Prometheus SoC: High-Level Synthesis of a RISC-V Softmax Accelerator System on the PYNQ-Z1

Jesse Levine
Department of Electronics
Carleton University
Ottawa, Canada

Abstract—This paper consolidates a multi-phase hardware design study into a single research narrative centered on Softmax acceleration for resource-constrained edge systems. Starting from a reproduced high-level synthesis (HLS) RV32I RISC-V processor, the work first extends the core to RV32IM, then optimizes the baseline through pipelining, and finally integrates a dedicated fixed-point Softmax accelerator into a shared-memory system-on-chip (SoC) called Prometheus SoC. The accelerator uses max-subtraction for numerical stabilization, piecewise-linear exponential and reciprocal approximations, and a four-lane vectorized datapath implemented with HLS dataflow. The resulting system is exposed to the processor through memory-mapped I/O and deployed on a Digilent PYNQ-Z1. Across the implementation-level comparison used for architecture selection, the software-only core scales as $287.89N + 74.34$ cycles, while the integrated SoC scales as $0.74N + 226.61$ cycles. This yields a measured speedup of $114.7\times$ at $N = 128$ and an asymptotic slope-based limit of approximately $389\times$. The accelerator also restores Softmax normalization accuracy, maintaining sums near unity while the software baseline degrades substantially with increasing vector length. FPGA measurements from the implemented SoC match simulation exactly at the sampled verification points and meet timing at 84.200 MHz. Together, these results justify the final architectural choice of a tightly integrated RISC-V plus accelerator SoC.

Index Terms—RISC-V, high-level synthesis, FPGA, Softmax, hardware accelerator, PYNQ-Z1, SoC

I. Introduction

Softmax is a small kernel in source-code size, but it is a disproportionately expensive kernel in hardware-constrained inference systems. Its computation requires exponentiation, accumulation, and normalization over an entire vector:

$$\sigma(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{N} e^{z_j}}. \tag{1}$$

On a small integer-oriented processor, these operations are instruction-bound and numerically fragile when implemented through fixed-point software approximations. At the same time, Softmax remains architecturally relevant because it is the final normalization step in classifiers and a core operation inside Transformer attention pipelines [1].

This project began from the HLS RV32I SoC of Toker [2], then evolved toward a specialized accelerator informed by recent approximation-based Softmax hardware for Transformer workloads [3]. The final result is not a standalone accelerator, but a complete RISC-V plus accelerator SoC that preserves processor control while offloading the numerically intensive Softmax path into dedicated hardware.

The main contributions of this work are:

1) a consolidated hardware progression from baseline RV32I reproduction to an RV32IM, pipelined, Softmax-capable system;
2) a fixed-point Softmax accelerator with log-sum-exp stabilization, piecewise-linear exponential and reciprocal approximations, and a four-lane vectorized datapath;
3) a shared-BRAM, MMIO-controlled SoC integration that allows a RISC-V core to launch and read back accelerator results without DMA; and
4) end-to-end validation on the PYNQ-Z1, including architecture-selection plots, board-level implementation data, and FPGA-versus-simulation agreement.

II. Related Work

Toker [2] demonstrates that a compact RV32I-based SoC can be described and implemented through HLS, providing the methodological starting point for this project. That work is important here because it shows that a soft RISC-V processor can be treated as a synthesizable HLS design object rather than only as handcrafted RTL. The first phase of this project reproduced that baseline and verified its fetch-decode-execute behavior on FPGA.

Kim et al. [3] focus instead on the non-linear bottlenecks of Transformer inference and show that approximation-based Softmax and LayerNorm acceleration can achieve strong speedups with controlled accuracy loss. Their work motivates the use of piecewise-linear approximation for the exponential and fixed-point-friendly normalization paths. The present paper differs in scope and target: instead of accelerating large Transformer layers on a high-end FPGA, it integrates a Softmax-specific accelerator into a small HLS RISC-V SoC on a Zynq-7020-class board.

The position of this work is therefore between those references. It begins from an HLS RISC-V system in the style of [2], then specializes that system around the Softmax bottleneck highlighted by [3], while keeping the processor in the loop as the software controller.

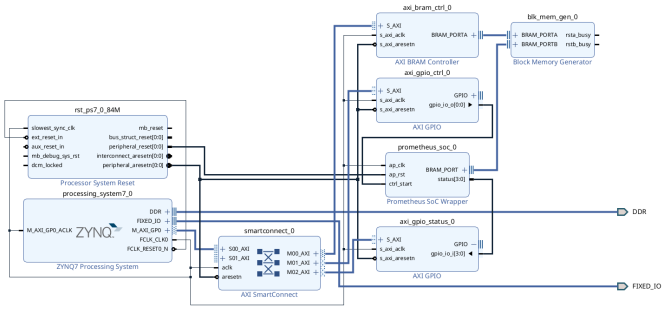


Fig. 1. Vivado block diagram of the implemented Prometheus SoC on the PYNQ-Z1. The Zynq processing system provides AXI control, while the programmable logic contains the packaged Prometheus SoC wrapper, AXI BRAM controller, shared BRAM, and GPIO-based start/status path.

III. Project Evolution and Final Architecture

The project evolved through five reports that directly informed the final system. Report 1 reproduced the RV32I baseline and established the bare-metal memory-image workflow. Report 2 extended the baseline to RV32IM so multiplication-based fixed-point kernels became practical. Report 3 proposed the specialization strategy: approximate exponentiation, memory-mapped control, and shared memory between processor and accelerator. Report 5 delivered the first pipelined RV32IM baseline and a standalone Softmax accelerator. Report 6 integrated these ideas into the final Prometheus SoC, which is the focus of this paper.

Fig. 1 shows the final deployed system. The Zynq processing system acts as host-side control logic and accesses the programmable logic through AXI SmartConnect. Shared BRAM holds the RISC-V program image, input logits, output probabilities, and debug words. Two AXI GPIO blocks implement a simple hardware-software handshake: one pulses the start signal, and the other returns latched status bits and FPGA cycle counts.

Inside the packaged HLS core, the RISC-V processor and accelerator share the same memory image. The software configures the accelerator through MMIO registers at address 0x7000, writing the input base address, output base address, debug base address, and vector length N, then setting the start bit. The accelerator reads logits from BRAM, computes exponentials after subtracting the vector maximum, accumulates the sum, computes an approximate reciprocal, and writes normalized probabilities back into BRAM. This integration avoids explicit data copying and keeps the processor responsible for orchestration rather than numerical throughput.

The exponential path exploits the identity

$$e^x = 2^{x/\ln 2}, \tag{2}$$

implemented with a fixed-point multiply by $1/\ln 2$, a power-of-two shift for the integer component, and a 17-point piecewise-linear lookup for the fractional component. The final accelerator version uses a four-word vector width and HLS dataflow so memory movement and arithmetic can overlap.

Measured across the project reports, the architecture progression is consistent: each stage traded additional area for a lower marginal Softmax cost. The important qualitative transition was from software execution on the pipelined core, to a standalone accelerator, and finally to an integrated SoC in which the processor remains the controller while the accelerator handles the numerically intensive kernel.

IV. Experimental Methodology

The evaluation combines data from the implementation-level comparisons used during architecture selection and the board-level FPGA validation performed on the final system. The software baseline is the pipelined RV32IM core from Report 5. The final system is the integrated Prometheus SoC deployed on the Digilent PYNQ-Z1 with the xc7z020clg400-1 device.

All Softmax inputs use fixed-point Q16.16 values. For the architecture-selection study, vector lengths from $N = 2$ to $N = 128$ are compared using the same workload set previously reported for the core-only and SoC implementations. For board validation, the SoC is swept from $N = 2$ to $N = 256$ through the JTAG/PYNQ execution flows stored in the repository, and the resulting FPGA cycle counts and normalization sums are recorded. The primary metrics are:

- clock cycles versus sequence length;
- Softmax probability sum as a numerical-stability proxy;
- implementation resource cost; and
- simulation-versus-FPGA agreement for the final SoC.

V. Results

A. Implementation-Level Architecture Selection

Table I captures the central design trade-off. The integrated SoC is substantially larger than the software-only baseline because it includes the accelerator datapath, extra local buffering, and the MMIO-based control structure. The architectural question is therefore whether this area increase is justified by the system-level throughput and accuracy gains.

Fig. 2 shows the throughput result that motivated the final architecture choice: the software-only baseline scales linearly but with a very large slope, while the integrated SoC keeps the per-element cost extremely small. Fig. 3 explains the corresponding numerical result. The software-only core does not merely run slower; its fixed-point approximation path causes the Softmax sum to collapse toward approximately 0.675 for longer vectors, which indicates significant accumulated numerical error. In contrast, the accelerator maintains sums close to 1.0 across the tested range because it uses explicit max-subtraction and a dedicated normalization path rather than long software

TABLE I
Implementation-level resource summary used for architecture selection. The baseline values come from the pipelined RV32IM implementation flow, while the SoC values come from the Prometheus SoC Vivado IP-flow implementation.

Resource / Metric	Core Only	Core + Accelerator SoC
LUT	2169	26090
FF	1465	52339
DSP	12	32
BRAM	0	9
Post-route period (ns)	11.345	11.514

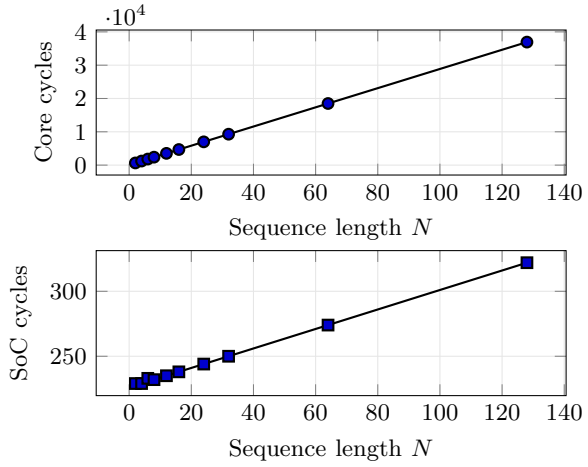


Fig. 2. Latency scaling of the software-only core and the integrated SoC on separate axes. A shared axis hides the SoC behavior because the software baseline is so much slower.

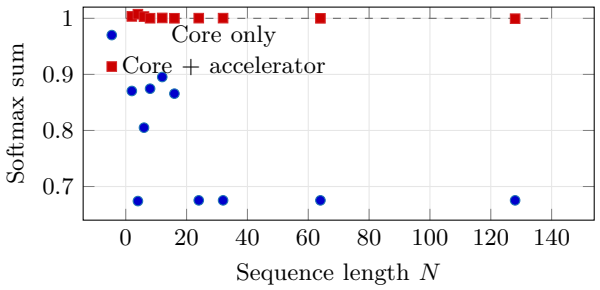


Fig. 3. Normalization comparison between the software-only core and the integrated SoC. The software path loses probability mass as N grows, while the accelerator maintains sums close to unity.

sequences for exponentiation and reciprocal evaluation. This is the first result that makes the final architecture selection credible: the accelerator is not only faster, it is materially better behaved numerically.

Fig. 4 shows that speedup grows rapidly with vector length because both systems scale linearly but with very different slopes. The measured ratio exceeds 114× at $N = 128$. Using the previously reported trend lines, the theoretical large-N speedup limit is approximately $287.89/0.74 \approx 389\times$. This asymptotic framing is useful because it explains why the SoC overhead at small N

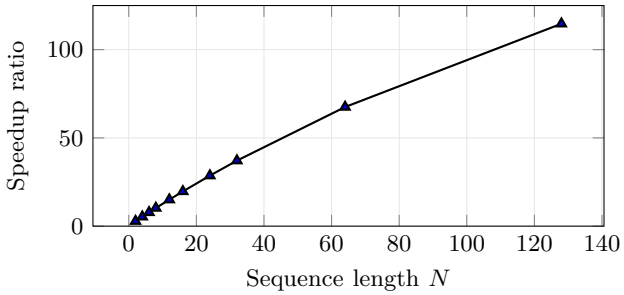


Fig. 4. Measured speedup of the integrated SoC over the software-only core. The slope ratio from the earlier implementation study implies an asymptotic upper bound of approximately 389×.

TABLE II
Implemented PYNQ-Z1 summary for the full Prometheus SoC. This table refers to the complete board-integrated design rather than only the packaged HLS IP block.

Metric	Implemented SoC on PYNQ-Z1
Board	Digilent PYNQ-Z1
FPGA	xc7z020clg400-1
Clock period (ns)	11.875
Clock frequency (MHz)	84.211
WNS (ns)	0.332
WHS (ns)	0.010
Slice LUTs	32717
Slice Registers	60358
Block RAM Tiles	68.5
DSPs	32
Timing status	Met

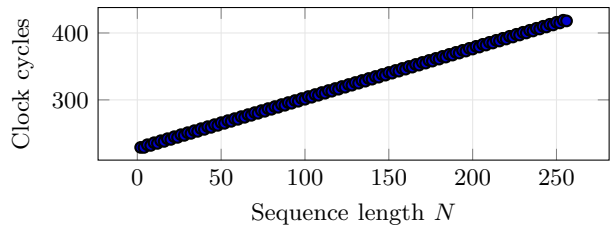
does not change the long-range architectural conclusion.

B. PYNQ-Z1 FPGA Implementation

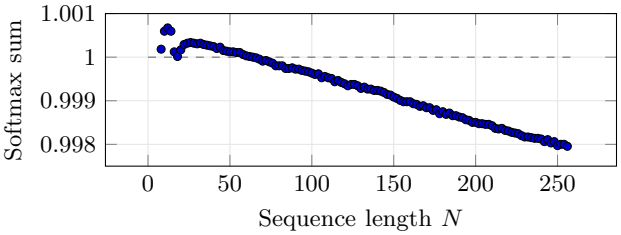
Table II shows that the full board-level design closed timing on the target FPGA. This matters because the final deliverable is not a simulation-only argument. The processor, shared BRAM, AXI glue logic, GPIO path, and accelerator were implemented together in a deployable system and exercised on hardware.

Fig. 5 confirms that the FPGA implementation preserves the intended system behavior across the full tested range. Latency remains low and grows gently with N, while the Softmax sum stays close to 1.0. Even at $N = 256$, the measured sum remains 0.997955, which is consistent with fixed-point quantization rather than an architectural failure.

The strongest end-to-end validation result is the simulation-versus-FPGA agreement at representative points. At $N = \{2, 8, 32, 64, 128\}$, the FPGA cycle counts match the simulated cycle counts exactly: 229, 232, 250, 274, and 322 cycles, respectively. The normalization sums also match to the printed precision, with the corresponding FPGA latencies ranging from 2.719 µs at $N = 2$ to 3.824 µs at $N = 128$. This validates not only the accelerator arithmetic, but also the correctness of the MMIO control path, shared-memory addressing, and integration wrapper.



(a) FPGA latency sweep.



(b) FPGA normalization behavior.

Fig. 5. Measured FPGA behavior of the implemented Prometheus SoC across the full sweep.

VI. Discussion

The combined evidence supports a clear architectural conclusion. A software-only implementation on a small RISC-V core is not an attractive endpoint for Softmax-heavy workloads because its per-element cost is too high and its fixed-point approximation path degrades normalization quality as vectors grow. The standalone accelerator already resolves most of the throughput problem, but the integrated SoC is the more useful system architecture because it preserves software programmability and embeds the accelerator in the same memory space as the processor.

The main limitation of the present design is scope rather than correctness. The integrated system accelerates Softmax only; it does not yet offload the earlier matrix-heavy layers of a full neural network, and it uses fixed-point arithmetic tuned for the tested workloads rather than a broad accuracy study over large benchmark suites. These are reasonable future directions. Another useful next step would be extending the accelerator interface beyond the current BRAM-centric deployment to a larger memory subsystem or DMA-capable integration.

VII. Conclusion

This paper consolidated the full project progression from a reproduced HLS RV32I core to a deployed RISC-V plus accelerator SoC. The final architecture, Prometheus SoC, integrates a four-lane fixed-point Softmax accelerator with a RISC-V controller through shared BRAM and MMIO. Relative to the software-only pipelined RV32IM baseline, the integrated system reduces the effective marginal Softmax cost from 287.89 cycles per element to 0.74 cycles per element, delivers a measured 114.7× speedup at $N = 128$, preserves normalization accuracy near unity, and matches simulation when exercised on the PYNQ-Z1 FPGA. These results make the final architecture selection straightforward: the publishable contribution is not only an accelerator, but a validated SoC-level demonstration of hardware-software co-design for Softmax acceleration on a constrained FPGA platform.

Appendix A
Summary of AI Usage

AI tools were used throughout the project as engineering assistants rather than as sources of experimental results. In the early phases, they were used to help interpret the reference RV32I architecture, understand HLS reports, and construct the bare-metal software flow used to generate memory-image test programs. During the accelerator and SoC phases, they were used to accelerate implementation work such as code drafting, testbench construction, script development, and explanation of synthesis and FPGA integration artifacts.

For this final paper, AI assistance was used to consolidate the prior reports into a single narrative, recover and cross-check stored datasets, generate and refine the LaTeX manuscript structure, and review consistency between the paper claims and the repository implementation. All architectural claims, numerical values, figures, and tables in the final manuscript were checked against the project source files, build artifacts, and measured outputs before inclusion.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in Advances in Neural Information Processing Systems, vol. 30, 2017.

[2] O. Toker, "A high-level synthesis approach for a RISC-V RV32I-based system on chip and its FPGA implementation," Engineering Proceedings, vol. 58, no. 1, p. 72, 2023, doi: 10.3390/ecsa-10-16212.

[3] R. Kim, D. Lee, J. Kim, J. Park, and S. E. Lee, "Hardware accelerator for approximation-based softmax and layer normalization in transformers," Electronics, vol. 14, no. 12, p. 2337, 2025, doi: 10.3390/electronics14122337.

[4] A. Waterman and K. Asanović, Eds., The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA, document version 20191214, RISC-V Foundation, 2019.